

Exploring layouts for the Picasa Gallery app using Flash Builder 4

In our last tutorial we created an application using the Flex SDK in Flash Builder 4 for browsing an image gallery exported from Picasa. Here, we'll explore the concept of layouts in Flex 4, and how we can take advantage of them.

Flex "spark" layouts

In a Flex application, we usually have multiple components that may be laid out horizontally, vertically, or in a combination of both. In the earlier version (v3) of the Flex SDK, you could use containers such as VBox for vertical layouts, and HBox for horizontal layouts. These both derived from the Box component. Flex 3 also supported a Canvas component that used absolute positioning, where instead of laying out components according to a particular layout, each component would be given a fixed position.

In the new component architecture called "spark" which has been adopted in Flex 4, the layout mechanism of containers is kept separate. So while the Group container, TitleWindow container, List container all have different features, and looks they can be assigned any layout, and the way they place components will be



Containers within containers, each with its own layout

derived from that. So while the Group and Panel components are simply for displaying components, and the List component is for displaying and navigating through data,

they can use the same layout.

You can write your own layout mechanisms if you want to control the placement. For example, you could take a list and create an iTunes-like coverflow, or a 3D carousel. Let us take our list of images from the last tutorial:

```
<s:List id="imagesList"
dataProvider="{index}"
width="100%" height="100%"
click="imagesList_
clickHandler(event)" >
<s:layout>
<s:TileLayout />
</s:layout>
<s:itemRenderer>
<fx:Component>
<mx:Image source="{data.
itemThumbnailImage}" />
</fx:Component>
</s:itemRenderer>
</s:List>
```

Here we've added the TileLayout component that manages the layout for the objects in our List within 'layout' property. The TileLayout component manages the arrangement of the images we load from the Picasa gallery. We can very easily swap this out and put in <s:VerticalLayout /> instead, and lay out items in a vertical orientation. We can configure our layouts as well, the TileLayout class, for example, allows you to configure horizontal and vertical gaps and alignments, the number of columns or rows, and you can select a row or column oriented layout.

Another interesting concept is that of virtual layouts. With a virtual layout you can have a list display hundreds or even thousands of elements, and the client runtime, Flash player will render only as many items as required. This will allow you to display a very large amount of data in Flex containers.

Due to this flexible architecture, it is not only possible for you to experiment with different layouts by changing just a single

tag in your code, but you can also change the layout of containers in your project at runtime!

Extending our Picasa Gallery app

We'll add a drop-down list to the UI that will list all the available layouts and on changing the selection in that list, the layout of our images will change.

This entails the following things:

1. We need to add a DropDownList component to our application
2. We need to create an array of layouts to display in the DropDownList
3. We need to associate each entry in the DropDownList with a layout object
4. We need change the layout with the selection

STEP 1: Add a DropDownList component to our application

This is simple, we just add the following after the list code in our previous application:

```
<s:DropDownList>
</s:DropDownList>
```

If you add this and run the application, you will see the drop down list we have added will overlay the earlier list. Since we have not specified a layout for our application, it defaults to using BasicLayout, which uses absolute positioning and layout, where you need to specify the position and size of every component added to the application.

This is quickly and simply remedied by adding a VerticalLayout component as the layout for the applications. We can do this by adding the following code inside the application tag:

```
<s:layout>
<s:VerticalLayout />
</s:layout>
```

To make the width of this drop-down list 100% and give it an id, add the following code: